UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingenieros de Telecomunicación
Grupo de Tratamiento de Imágenes

Proyectos en Ingeniería de Datos y Sistemas (PIDS)

Proyecto 2

Sistema de detección de vehículos y conducción autónoma

MEMORIA DEL PROYECTO FASES 1 Y 2

Grupo	2
Autores	Javier Saguar Alejandro Cuevas Mónica Fernández Pedro José Orrego Daniel Naval
Entrega	Entrega 2 · Bloques 1 y 2
Fecha de entrega	25/10/2026

Reparto del trabajo

Integrante	Tareas del Bloque 1
Javier Saguar	Lote A (vídeos 01-02): extracción, anotación y revisión cruzada. Fichas de los vídeos, criterios de anotación y depuración del dataset. Aumento de datos sobre el script de Alejandro, que mejoró y ejecutó. Redacción de la memoria.
Alejandro Cuevas	Lote B (vídeos 03-04): extracción, anotación y revisión cruzada. Análisis exploratorio de los datos (EDA) y script de entrenamiento sobre el que se integró el aumento. Redacción de la memoria.
Mónica Fernández	Lote C (vídeos 05-06): extracción, anotación y revisión cruzada. Calibración entre anotadores y revisión de la memoria.
Pedro José Orrego	Lote D (vídeos 07-08): extracción, anotación y revisión cruzada. Calibración, selección de figuras del EDA y diseño de la web de figuras interactivas (GitHub Pages).
Daniel Naval	Lote E (vídeos 09-10): extracción, anotación y revisión cruzada. Calibración y redacción de la memoria.

Tabla 1. Reparto del trabajo del Bloque 1 entre los cinco integrantes.

Índice

Reparto del trabajo 1
Índice 2
1. Introducción 3
3. Anotación de datos 4
4. Análisis exploratorio de los datos 5
5. Aumento de datos 8
6. Depuración de datos 11
7. El pipeline de entrenamiento 12
8. Modelos y transfer learning 12
9. Hiperparámetros 13
10. Métricas 14
11. Experimentos 15
12. Análisis cualitativo 20
Referencias 22

1. Introducción

1.1. Contexto y motivación

El proyecto consiste en desarrollar un detector de robots RoboMaster rivales a partir de las imágenes captadas por la cámara de un robot móvil. Esta tarea presenta varias dificultades, ya que el movimiento del robot, los cambios de tamaño aparente, las condiciones de iluminación y la presencia de obstáculos pueden afectar a la localización de los robots. La detección obtenida servirá como base para desarrollar posteriormente una estrategia de apuntado y actuación.

1.2. Objetivos del proyecto

El trabajo se divide en tres bloques. El primero se centra en la creación y el análisis del conjunto de datos, el segundo en el entrenamiento y ajuste del detector, y el tercero en su integración con el robot. En este primer bloque se obtuvieron 808 imágenes originales anotadas, de las cuales finalmente se seleccionaron 796 tras revisar las etiquetas y comprobar su calidad.

2. Recolección de datos

2.1. Extracción de imágenes

La cámara utilizada genera vídeo a una resolución de 1280 × 720 píxeles y una frecuencia aproximada de 30 fps. Para evitar trabajar con imágenes demasiado similares, se estableció un intervalo de extracción de 24 fotogramas (INTERVAL=24), equivalente a una imagen cada 0,79 segundos aproximadamente. Aplicando este criterio a los diez vídeos se obtuvieron 808 imágenes a partir de un total de 19.508 fotogramas y 643,8 segundos de grabación. El número de imágenes se calculó por separado para cada vídeo mediante floor(frames/24), sin redondear previamente la duración total.

Vídeo	Escenario	Duración s	Frames	Imágenes
V01	Vestíbulo ETSIT	65,4	1982	82
V02	Aula A-201.L	59,9	1815	75
V03	Aula A-201.L	60,2	1823	75
V04	Exterior edificio A	66,1	2003	83
V05	Pasillo A-201.L	59,6	1807	75
V06	Vestíbulo ETSIT (mesas)	74,1	2244	93
V07	Vestíbulo ETSIT	61,2	1855	77
V08	Vestíbulo ETSIT (mesas)	81,2	2460	102
V09	Exterior edificio A	56,2	1704	71
V10	Aula A-201.L	59,9	1815	75
Total	1280 × 720 a 30,30 fps	643,8	19 508	808

Tabla 2. Grabaciones propias y número exacto de imágenes extraídas.

3. Anotación de datos

Se utilizó labelImg [1] y XML PascalVOC, con una única clase robot. Cada integrante anotó su lote y otro revisó las cajas. Se conservaron imágenes negativas con XML vacío para que el entrenamiento no las ignore.

Caso	Criterio operativo
Robot rival	Incluir chasis, ruedas, gimbal y cañón; excluir antenas.
Elementos ajenos	No anotar el cañón propio, reflejos ni robots en pantallas.
Oclusión y borde	Anotar si es reconocible y se ve al menos un 30 %; caja sobre la parte visible.
Difficult	Marcar si tiene menos de 30 px de ancho, menos de la mitad visible o está muy movido.
Robot muy pequeño	No anotar por debajo de 10 px de lado; conservar criterios en el aumento.
Negativo	Guardar imagen y XML sin objetos.

Tabla 3. Criterios comunes aplicados a los lotes.

Como se observa en la figura siguiente, el criterio distingue cajas normales y difficult sin crear una segunda clase. Una frontera superior distinta alteraría tamaños y centros, por lo que se excluyen antenas y se ajusta el resto del robot de manera consistente.

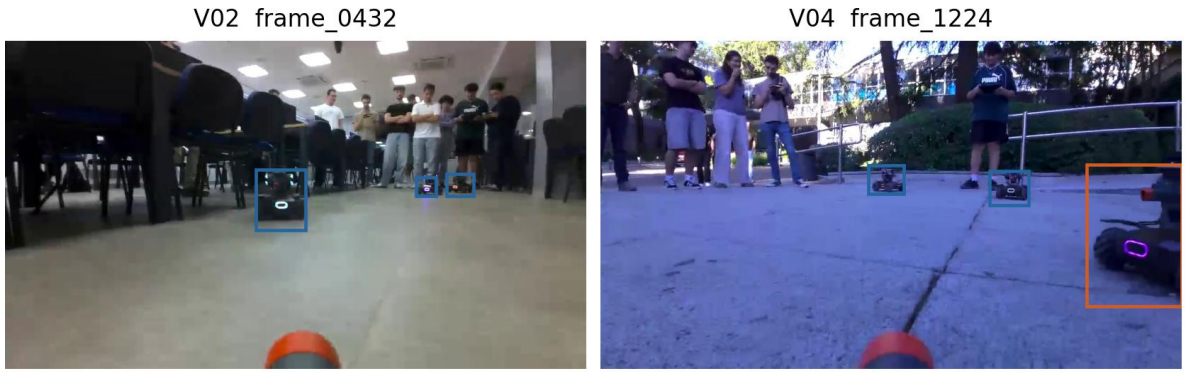


Figura 1. Anotaciones de interior y exterior. Azul: normal; naranja: difficult.

Los cinco anotadores etiquetaron las mismas 20 imágenes de calibración. El IoU medio entre parejas fue 0,81; el borde superior presenta la mayor discrepancia, 7 px de mediana frente a 4–5 px en los otros bordes. Esto se debe a que la parte superior del robot es la más ambigua de anotar, debido a las antenas.

4. Análisis exploratorio de los datos

4.1. Inventario y equilibrio

El EDA se calcula sobre los 808 originales, antes de excluir imágenes. Los aumentos no se mezclan en estas estadísticas. Las cifras de la tabla y la figura permiten separar cantidad de imágenes, número de objetos y dificultad.

Estadística	Valor
Secuencias / imágenes	10 / 808
Imágenes medias por secuencia	80,8
Cajas / cajas medias por imagen	1420 / 1,76
Difficult	375 cajas · 26,4 %
Imágenes negativas	41 · 5,07 %
Área media de caja	33,627 miles de píxeles²
Área mínima / máxima	1,020 / 661,074 miles de píxeles²
Aspecto mediano ancho / alto	1,11

Tabla 4. Estadísticas de los originales anotados.

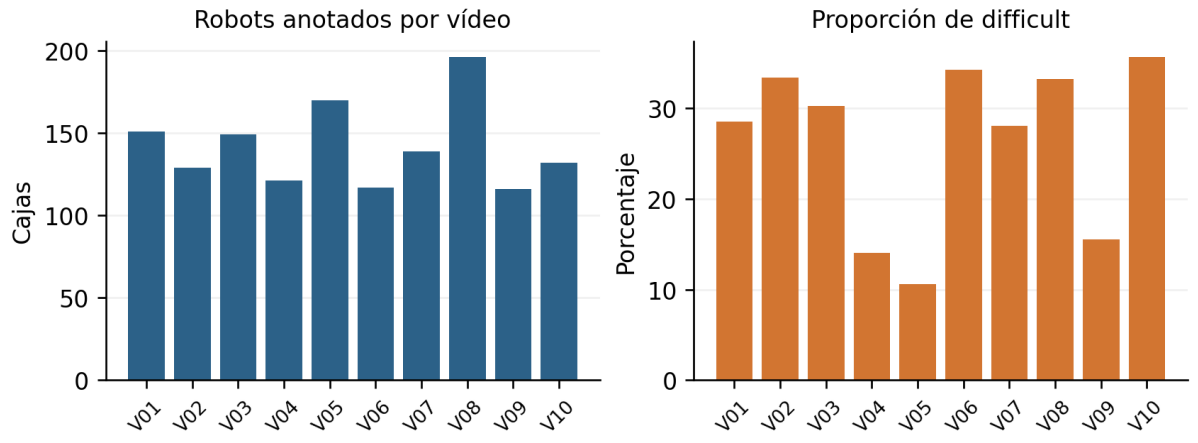


Figura 2. Recuento y dificultad por vídeo; sus diferencias motivan analizar cada secuencia.

No hemos intentado que todos los conjuntos sean exactamente iguales. Por ejemplo, V05 registra 170 robots, mientras que V06 contiene 117. Además, V06 incluye 22 ejemplos negativos, mientras que V08 no tiene ninguno y tiene 65 casos especialmente difíciles.

4.2. Tamaño forma e iluminación

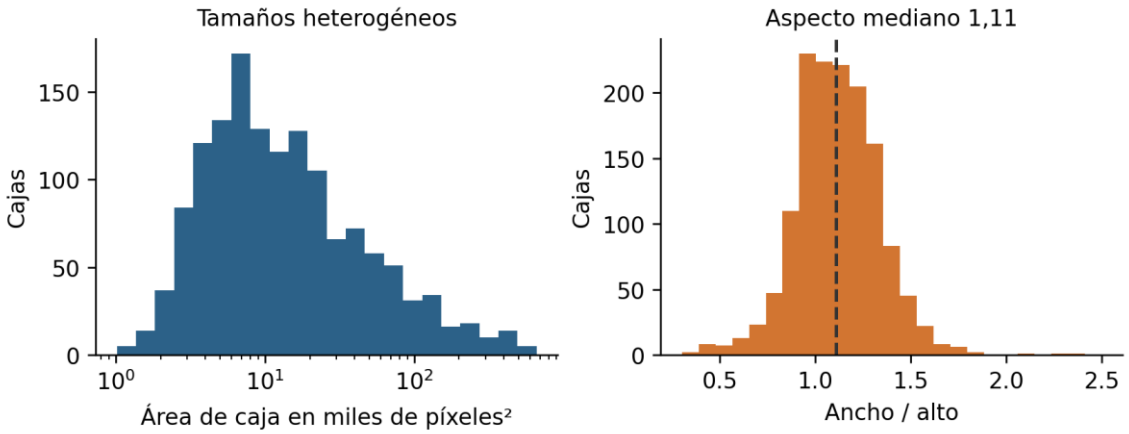


Figura 3. Área y forma de las cajas de los 1420 robots. El eje de área es logarítmico.

La distribución de la figura tiene una cola amplia de áreas, pero no hay cajas originales de menos de 30 px de ancho. La relación de aspecto mediana es 1,11. El análisis de cobertura de anclas obtiene un 99,6 % con IoU ≥0,5: no se cambia su configuración por ahora y se evita deformar las proporciones durante el aumento.

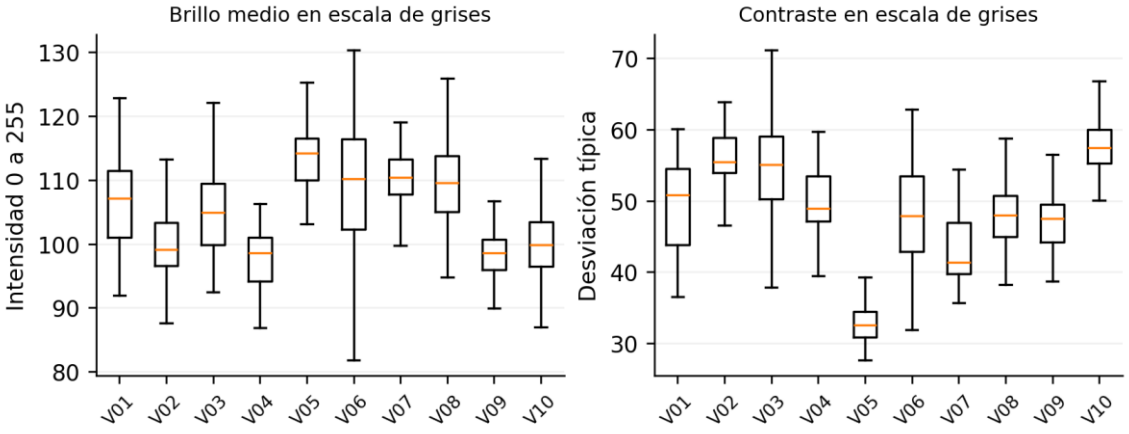


Figura 4. Brillo y contraste por secuencia, calculados en escala de grises.

Los canales RGB reflejan el color predominante de cada imagen y no solo las condiciones de iluminación, ya que también están influidos por elementos como el suelo o las paredes. Por ello, se complementan con medidas de intensidad y nitidez. Además, el tono azulado habitual en exteriores justifica aplicar ajustes de color moderados para no confundirlo con una iluminación deficiente. Se identificaron 52 imágenes poco nítidas para revisión visual.

La miniatura más parecida a una imagen procede del mismo vídeo en el 74 % de los casos. Esa correlación aconseja separar grabaciones completas, evitando una validación artificialmente fácil con fotogramas vecinos.

4.3. Posición y evolución temporal

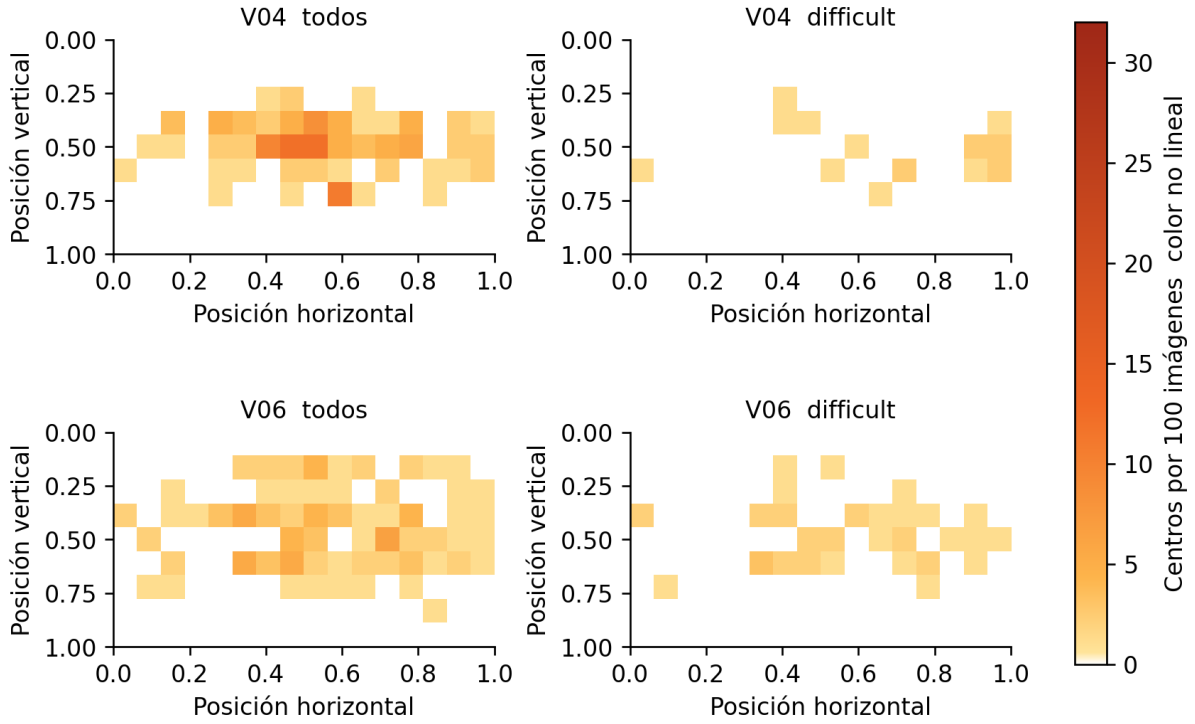


Figura 5. Gráficas del proyecto. Versión interactiva disponible en [6]: https://javiersaguar.github.io/PIDS-Proyecto2/

La figura compara la posición de las anotaciones en el exterior (Video 4) y el interior (Video 6) con muchos negativos. Se normalizan los centros por tamaño de imagen y el histograma 16 × 9 por número de imágenes. Aunque la mayoría de cajas están cerca del centro, la proporción de cajas cercanas a los bordes es mayor en las difficult.

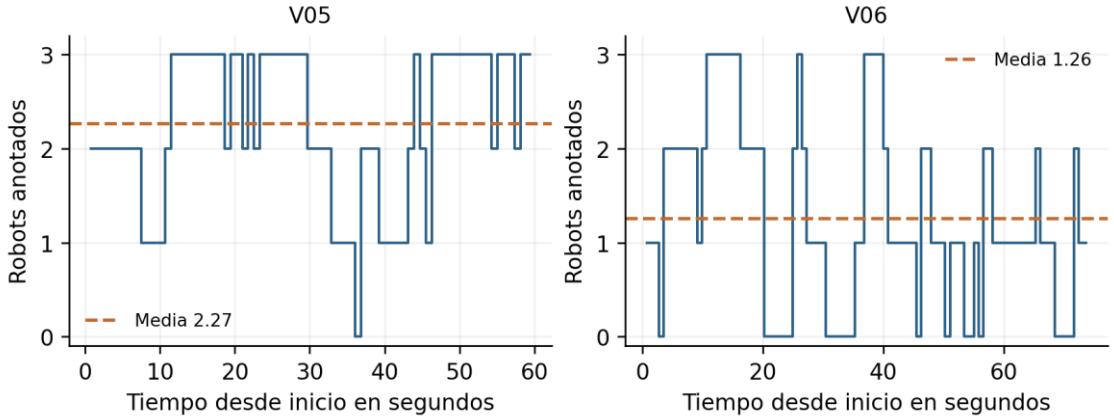


Figura 6. Gráficas del proyecto. Versión interactiva disponible en [6]: https://javiersaguar.github.io/PIDS-Proyecto2/

Los gráficos muestran información de los videos, que segmentos tienen más y menos robots en pantalla, pero también permiten identificar discrepancias en las anotaciones, cuando se mueven los robots (causando que no se marquen por estar demasiado movidos u obstruidos). Entorno al segundo 20 de V05, por ejemplo, parece que hubiera 3 robots en pantalla, pero no se anotaron en algunos frames debido probablemente a que estaban borrosos o tapados.

5. Aumento de datos

El aumento se aplica solo al conjunto de entrenamiento y siempre a partir de los originales, nunca sobre copias ya aumentadas. Cada una de las 796 imágenes depuradas recibe tres copias, lo que da 3184 imágenes y 5566 cajas. Validación (672 imágenes) y test (718) son secuencias de otros grupos y se copian sin transformar, porque deben parecerse a lo que verá el robot y no a nuestras transformaciones.

El criterio para elegir cada transformación es doble, debe simular una variación que el robot puede encontrar en la demo y que nuestros diez vídeos no cubren del todo, y no debe producir escenas imposibles ni cajas que dejen de ajustarse al robot. Cada copia recibe una combinación distinta, con una media de 2,7 transformaciones, y la intensidad de cada una se sortea dentro de su rango.

5.1. Transformaciones geométricas

Transformación	Rango y probabilidad	Intención
Volteo horizontal	p = 0,5	Un robot visto en espejo sigue siendo un robot válido y el EDA muestra que los centros de las cajas son simétricos en horizontal. Duplica las orientaciones sin crear escenas irreales.
Escala sin deformar	×0,6–1,25	Alejar produce robots más pequeños y simula mayor distancia: no había cajas originales de menos de 30 px de ancho y la demo puede exigir detectar más lejos. Acercar produce robots cortados por el borde, frecuentes en nuestros vídeos.
Desplazamiento	±8 % horizontal, ±5 % vertical	Cambia la posición del robot sin sacarlo de la franja a media altura en la que aparece siempre.
Giro	±6°	La cámara se inclina al acelerar y en V06; el EDA atribuye a esa inclinación las cajas atípicas. Con giros mayores la caja deja de ajustarse al robot.

Tabla 5. Transformaciones geométricas utilizadas y su intención.

Se aplican dos decisiones de implementación para evitar errores silenciosos. El hueco que deja un giro o un alejamiento se rellena de gris y no con un reflejo de la imagen, que podría duplicar un robot en el borde sin su caja y enseñar a la red a tratarlo como fondo. Al girar, la caja se recalcula con el método ellipse, que la mantiene ajustada; el método por esquinas la agranda (IoU de 0,85 con ±6°).

5.2. Color e iluminación

Transformación	Rango y probabilidad	Intención
Brillo y contraste	±20 %; p = 0,5 conjunta con gamma	Cubre iluminaciones no grabadas: otra hora del día, luces apagadas o contraluz.
Gamma	75–125	Complementa al brillo: aclara u oscurece las sombras sin saturar las zonas iluminadas.
Tono y saturación	±8 de tono, ±20 de saturación; p = 0,3	Cambios suaves, como recomienda el EDA por el tono azulado de los exteriores. Un cambio fuerte alteraría el color de los LED, que es la pista más clara para reconocer el robot.
Desplazamiento RGB	±12 por canal; alternativa al tono	Simula un balance de blancos distinto de la cámara.

Tabla 6. Transformaciones de color e iluminación utilizadas y su intención.

5.3. Calidad de imagen

De estas cuatro transformaciones se aplica como mucho una por copia (p = 0,3), para no acumular degradaciones que harían la imagen peor que cualquier fotograma real. El desenfoque de movimiento se sortea aparte.

Transformación	Rango y probabilidad	Intención
Desenfoque de movimiento	Núcleo de 3–9 px; p = 0,15	Es el defecto real de nuestros vídeos, porque se mueven la cámara y los robots. Sustituye al desenfoque uniforme del notebook original.
Compresión JPEG	Calidad 35–75	El vídeo llega comprimido desde el robot y pierde calidad cuando la señal es mala.
Ruido de sensor	Gaussiano (varianza 10–40) o ISO	Reproduce el ruido que aparece con poca luz.
Menor resolución	×0,5–0,75	El SDK puede entregar el vídeo a menos resolución que la aplicación del robot.

Tabla 7. Transformaciones de calidad de imagen utilizadas y su intención.

5.4. Transformación propia: oclusión sintética

En la demo los robots estarán detrás de obstáculos fijos, y en nuestros vídeos ya aparecen detrás de patas de silla y de piernas. Para entrenar con más casos parcialmente tapados se dibujan una o dos barras oscuras casi verticales en el 15 % de las copias. Ninguna barra puede tapar más del 35 % del ancho de un robot, de modo que sigue siendo reconocible y su caja no cambia, como establece la regla R4 de anotación para obstáculos finos.

Se prefiere a CoarseDropout, que coloca rectángulos al azar y puede tapar un robot entero dejando su caja sobre el fondo. Es la transformación menos realista del conjunto, ya que una barra lisa no es una pata de silla; por eso se deja preparada una variante con postes texturados (apartado 5.6).

5.5. Transformaciones descartadas

Transformación	Motivo del descarte
Volteo vertical	El robot nunca está boca abajo y el suelo nunca aparece arriba.
Giros de 90°	La escena tiene una orientación fija; el robot no aparece de lado.
Perspectiva	En Albumentations 1.4.18 la caja devuelta no coincide con el robot (IoU de 0 en la prueba automática). Introduciría cajas falsas en el entrenamiento.
Recorte 640 × 480 redimensionado a 1280 × 720	Es el aumento del notebook original. Pasa de 4:3 a 16:9 y ensancha los robots un 33 %; sobre nuestras imágenes pierde además el 14,1 % de los robots. Se sustituye por la escala sin deformar.
Desenfoque uniforme de 7 px al 50 %	El 27 % de los robots ya es difficult, casi siempre por movimiento. Desenfocar la mitad de las copias haría el entrenamiento más borroso que la realidad, y el desenfoque uniforme no se parece al de movimiento.
Intercambio de canales	Produce colores que la cámara nunca dará.

Transformación	Motivo del descarte
Niebla, lluvia, destellos y sombras (perfil base)	No son condiciones de la demo, que es en interior. Sombras y lluvia se exploran en perfiles separados (apartado 5.6).
Mosaico y copy-paste	Son útiles con muchos objetos pequeños; aquí hay 1,8 robots por imagen y la escala ya aporta robots más pequeños.

Tabla 8. Transformaciones descartadas y motivo.



Figura 7. Original y una copia del aumento base, con las cajas transformadas.

Se descartan volteo vertical y giros de 90° (porque solo vamos a usarlo para detectar robots de pie), el recorte 4:3 redimensionado a 16:9 (porque ensancha robots). El relleno gris evita duplicar robots sin etiqueta en los bordes. Se usan cajas ajustadas y filtros de lado mínimo 10 px y visibilidad 30 %

El notebook ejecutado con el aumento de datos está disponible en [8].

6. Depuración de datos

Se revisaron visualmente 69 imágenes para comprobar su calidad y utilidad. De ellas, 52 presentaban poca nitidez, 11 eran casi duplicadas y 6 planteaban dudas. Al final, descartamos una imagen en la que aparecía un robot que no podía anotarse de forma consistente y se eliminaron otras 11 por ser redundantes. En cambio, se mantuvieron 41 imágenes movidas con sus robots marcados como *difficult*, así como los negativos difíciles. Esta decisión se tomó porque una imagen poco nítida no tiene por qué ser inútil para el entrenamiento, pues también permite representar las condiciones reales en las que trabajará el detector.

Conjunto	Imágenes	Cajas
Original anotado	808	1420
Original tras depuración	796	1399
Copias del aumento base	2388	4167
Entrenamiento total	3184	5566

Tabla 9. Efecto de la depuración y del aumento base.

6.1. Ampliaciones preparadas y límites

Se prepararon además, copias aumentadas con sombras y llovizna sintética en los vídeos (V04 y V09), obstáculos con textura y más peso de las imágenes con difficult, para que el modelo aprendiera con degradaciones visuales y fuera más robusto. Al entrenar con este perfil, el AP@0,5 en validación apenas cambió (0,9426 frente a 0,9433 con el aumento base), sin diferencia en difficult (0,6471 frente a 0,6462). Una posible explicación es que el aumento base ya cubría parte de esa variabilidad, pero no se ha comprobado por falta de explicabilidad su causa raíz.

7. El pipeline de entrenamiento

El entrenamiento se realiza con PyTorch [18] y torchvision [4] utilizando 3184 imágenes (las 796 originales y tres versiones aumentadas de cada una). La validación incluye 672 imágenes de 11 secuencias grabadas, con 1561 robots y 255 difficult. Por su parte, el test contiene 718 imágenes de 13 secuencias externas, con 1721 robots y 270 difficult. Este conjunto se reserva para evaluar cada modelo una sola vez, sin utilizar sus resultados para tomar decisiones (apartado 11.5).

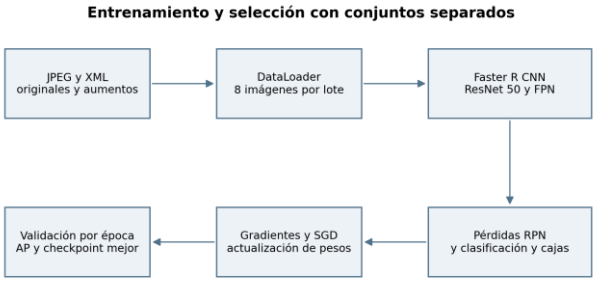


Figura 8. Flujo usado para ajustar pesos y seleccionar el checkpoint con validación.

La Figura 8 muestra el flujo de entrenamiento. Los JPEG y XML, tanto originales como aumentados, pasan al Dataset, que carga cada muestra como una imagen RGB en formato tensor con valores entre 0 y 1 y un diccionario con sus cajas y etiquetas. El DataLoader agrupa las imágenes en batches de 8 y se las entrega al modelo, que las normaliza y escala internamente. Durante el entrenamiento, el modelo devuelve las pérdidas, su clasificación y regresión, cuya suma constituye la función de pérdida. Después, la retropropagación calcula los gradientes y SGD los utiliza para actualizar los pesos.

Un batch es el grupo de imágenes procesadas en cada paso, mientras que una época corresponde a una pasada completa por el conjunto de entrenamiento. Se trabaja por batches porque las 3184 imágenes no caben a la vez en la memoria de la GPU y porque así se actualizan los pesos varias veces por época.

Al terminar cada época, se evalúa el modelo con las 672 imágenes de validación, procedentes de secuencias que no ha visto, sin actualizar los pesos (no_grad). Se calculan la pérdida de validación y el AP para medir su capacidad de generalización y elegir el mejor checkpoint. Es importante que esta fase no modifique los pesos para mantener una evaluación independiente. Finalmente, model_last se guarda siempre, mientras que model_best solo se guarda cuando mejora el AP@0,5.

Las cajas difficult también participan en el entrenamiento: sin_difficult elimina el flag del diccionario que se entrega al detector, no sus cajas.

8. Modelos y transfer learning

Un detector está formado por un backbone, que es la red convolucional (ResNet-50 [11], MobileNetV3 [13] o VGG16 [14]) encargada de extraer características de la imagen, y el resto de componentes, que se ocupan de localizar y clasificar los objetos. Entre ellos están el cuello FPN [12], que combina la cabeza propuesta, la clasificación y su regresión. Se distingue el backbone porque es el componente que se puede intercambiar entre modelos, influye mucho en el coste computacional y cuenta con pesos preentrenados.

Todos los modelos utilizan transfer learning, es decir, parten de pesos aprendidos con COCO [15], una base de datos general con más de cien mil imágenes anotadas, y sustituyen la cabeza de clasificación por otra con dos clases: fondo y robot. De esta forma, la red aprovecha lo aprendido sobre bordes, texturas y formas, y adapta sus predicciones al objeto que queremos detectar [3,4]. Esto resulta especialmente importante porque solo disponemos de 796 imágenes originales de diez vídeos, una cantidad insuficiente para entrenar desde cero redes con decenas de millones de parámetros. Además, permite alcanzar el mejor checkpoint en pocas épocas (apartado 11.3).

Modelo	Tipo	Cómo funciona	Backbone y cuello
Faster R-CNN v1	Dos etapas	Primero propone regiones con posibles objetos (RPN) y después las clasifica y ajusta sus cajas.	ResNet-50 + FPN
Faster R-CNN v2	Dos etapas	Sigue el mismo proceso, pero incorpora mejoras en las capas de predicción y en el entrenamiento [16].	ResNet-50 + FPN
Faster R-CNN MobileNet	Dos etapas, versión ligera	Utiliza la misma idea, pero con una red más ligera para reducir el coste computacional y priorizar la velocidad.	MobileNetV3-L + FPN
RetinaNet v1 / v2	Una etapa	Detecta y clasifica objetos directamente en distintas posiciones y escalas. Utiliza la pérdida focal para dar más importancia a los ejemplos difíciles frente al gran número de regiones de fondo [9]. La v2 incorpora mejoras en la cabeza y el entrenamiento.	ResNet-50 + FPN
SSD300	Una etapa	Detecta objetos de distintos tamaños en una imagen de entrada de 300 × 300 píxeles [10].	VGG16
SSDlite320	Una etapa, versión ligera	Utiliza convoluciones más eficientes para reducir el coste computacional y procesa imágenes de 320 × 320 píxeles [17].	MobileNetV3-L

Tabla 10. Los siete modelos comparados.

9. Hiperparámetros

Parámetro	Valor aplicado
Épocas y tamaño (lote)	30 épocas; lote efectivo de 8 imágenes. En res1024, microbatch de 4 × 2.
Semilla aleatoria	0
Optimizador	SGD, con momentum de 0,9 y regularización (weight decay) de 0,0005.
Tasa de aprendizaje	0,0001 en los modelos originales y 0,001 en lr001 y res1024.
Ajuste de la tasa	ReduceLROnPlateau, según la pérdida de validación.
Reducción y paciencia	Multiplicador ×0,1 tras 5 épocas sin mejora. Valor mínimo: 1e−10.
Resolución (entrada)	800/1333 píxeles; en res1024, 1024/1820.
Datos de entrenamiento	Imágenes originales y tres copias aumentadas de cada una, aplicando el mismo método de aumento base.

Selección del modelo	Se conserva el checkpoint con mayor AP@0,5 en validación.
Entorno de ejecución	PyTorch 2.4.1, torchvision 0.19.1 y GPU NVIDIA RTX 3060 de 12 GB.

Tabla 11. Configuración común de los experimentos medidos.

La Tabla 11 recoge los hiperparámetros comunes utilizados en los experimentos. Estos valores controlan cómo aprende el modelo y permiten comparar las distintas arquitecturas en condiciones similares.

Batch size (tamaño del lote). Indica cuántas imágenes se procesan antes de actualizar los pesos. Un lote pequeño puede producir gradientes más variables, mientras que uno grande requiere más memoria. Durante las pruebas, algunos modelos no cabían en la GPU con un lote de 8 imágenes. Por ello, Faster R-CNN v2 y res1024 utilizan microlotes de 4 imágenes con dos acumulaciones de gradientes, manteniendo un lote efectivo de 8. Se fijó este valor para comparar las arquitecturas, por lo que no se midió directamente el efecto de cambiar el tamaño del lote.

Learning rate (tasa de aprendizaje). Determina el tamaño de los pasos que se dan al actualizar los pesos. Una tasa demasiado alta puede hacer que el entrenamiento se vuelva inestable y empeore los resultados. En las pruebas, con 0,0001 el mejor checkpoint se alcanza en la época 29, mientras que con 0,001 se alcanza en la 5. Con 0,002, el máximo llega en la época 1 o 2, pero después el AP disminuye entre 1,1 y 1,4 puntos. Con 0,0005, el máximo se alcanza entre las épocas 5 y 14. Estos resultados muestran que una tasa mayor no garantiza un mejor modelo. El programador reduce la tasa cuando la pérdida de validación deja de mejorar, aunque en las pruebas no siempre lo hace a tiempo para recuperar el mejor resultado (apartado 11.1).

Épocas. Una época corresponde a una pasada completa por las imágenes de entrenamiento. Aunque se fijó un máximo de 30, no siempre es necesario llegar hasta el final. En seis de los siete modelos, el mejor checkpoint se obtiene entre las épocas 3 y 7; en SSD300 se alcanza en la 12. Además, la pérdida de validación llega a su mínimo en la época 4 y después aumenta, mientras la de entrenamiento sigue disminuyendo. Esto indica que el modelo puede empezar a sobreajustarse. Por eso, se selecciona el mejor checkpoint de validación en lugar de asumir que el último es el mejor.

Optimizador y programador. Se utiliza SGD para actualizar los pesos en función de los gradientes calculados durante el entrenamiento. El parámetro momentum (0,9) aprovecha parte de la dirección de las actualizaciones anteriores para suavizar las variaciones, mientras que weight decay (0,0005) penaliza los pesos excesivamente grandes y ayuda a limitar el sobreajuste. Por su parte, ReduceLROnPlateau reduce la tasa de aprendizaje por un factor de 10 cuando la pérdida de validación no mejora durante cinco épocas. Así, el entrenamiento puede avanzar con pasos mayores al principio y realizar ajustes más pequeños cuando deja de mejorar. No se probaron otros optimizadores, por lo que no se puede afirmar que SGD sea superior a las alternativas.

10. Métricas

El IoU mide cuánto se solapan la caja predicha y la caja real, dividiendo su intersección entre su unión. Para evaluar las detecciones, se exige un IoU ≥ 0,5 y una confianza ≥ 0,5. Las detecciones se emparejan

por confianza, es decir, si una coincide con un robot anotado, es un verdadero positivo (TP); si no tiene pareja, es un falso positivo (FP); y si un robot anotado queda sin detectar, es un falso negativo (FN).

La precisión mide qué proporción de las detecciones son correctas (TP/(TP+FP)), mientras que la recall indica qué proporción de los robots anotados se detecta (TP/(TP+FN)). El F1 combina ambas mediante su media armónica. El score_threshold establece la confianza mínima para aceptar una detección, al subirlo, suelen disminuir los FP y aumentar la precisión, pero también pueden aumentar los FN y bajar la exhaustividad. Al reducirlo, ocurre lo contrario. Este es el compromiso entre ambas métricas (apartado 11.5).

La AP (Average Precision) resume la curva precisión-exhaustividad al variar el umbral de confianza y se calcula como el área bajo esa curva. AP@0,5 utiliza un IoU de 0,5, mientras que AP@0,75 exige una mayor precisión. AP 0,50:0,95 promedia los resultados para umbrales de IoU entre 0,50 y 0,95; es la métrica conocida como mAP en COCO. La «m» indica que se promedia entre clases y, al trabajar con una sola clase, la mAP coincide con la AP.

Por último, las anotaciones incompletas en validación pueden hacer que detecciones correctas se contabilicen como falsos positivos. Por tanto, estos resultados deben interpretarse con cautela, ya que no todos los falsos positivos representan errores visuales confirmados.

11. Experimentos

Los experimentos se organizaron en tres fases. Primero se ajustaron los hiperparámetros con un único modelo, Faster R-CNN v1 (el modelo de partida que propone el enunciado), para no multiplicar el coste de entrenamiento, se comparó el aumento de datos (sin aumento frente al aumento base), la tasa de aprendizaje (0,0001 y 0,001) y la resolución de entrada (apartados 11.1 y 11.2). Después, con los hiperparámetros ya fijados, se compararon las siete arquitecturas (11.3) y se estudió la tasa de aprendizaje con las tres mejores (11.4). Por último, se evaluó cada modelo en test una sola vez (11.5).

11.1. Aumento de datos y tasa de aprendizaje Faster R-CNN v1

Experimento	Mejor época	AP50 historial	AP50 evaluación	AP evaluación
Sin aumento lr 0,0001	20	0,9229	0,9228	0,5809
Aumento base lr 0,0001	29	0,9425	0,9416	0,6081
Aumento base lr 0,001	5	0,9432	0,9433	0,6302
lr 0,001 res1024	2	0,9464	0,9465	0,6047

Tabla 12. Mejor checkpoint por AP50 en validación.

La Tabla 12 compara los cuatro primeros experimentos. El aumento de datos mejora especialmente la detección de objetos difficult, cuyo AP50 pasa de 0,5518 a 0,6341. Sin embargo, al multiplicar por cuatro las imágenes y actualizaciones, no se puede atribuir toda la mejora únicamente a las transformaciones. Por otro lado, subir la tasa de aprendizaje de 0,0001 a 0,001 mejora el ajuste de las cajas (AP 0,50:0,95: +2,21 puntos; AP75: +3,14), apenas cambia el AP50 (+0,17) y permite alcanzar el mejor resultado en la época 5, frente a la 29.

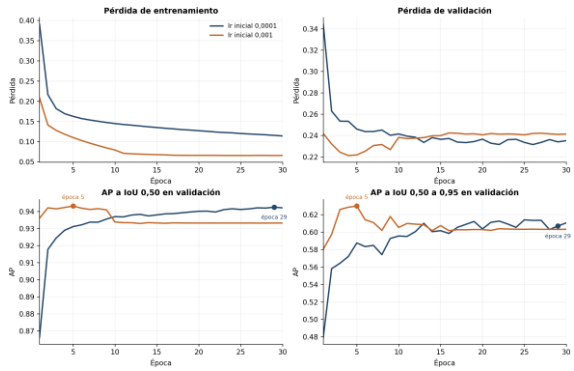


Figura 9. Pérdidas y AP por época. Los puntos señalan los checkpoints elegidos por AP50.

Con un learning rate de 0,001, la pérdida de validación alcanza su mínimo en la época 4 (0,2212) y después aumenta hasta 0,2413, mientras que la pérdida de entrenamiento sigue bajando hasta 0,0657 (Figura 9). Esto indica un posible sobreajuste, el modelo sigue aprendiendo de las imágenes de entrenamiento, pero empeora en validación. Por ello, se conserva model_best en lugar de model_last. Como muestra la Figura 10, ReduceLROnPlateau reduce la tasa de aprendizaje de 0,001 a 1e−7 en las épocas 11, 17, 23 y 29, pero no consigue recuperar el mejor resultado.

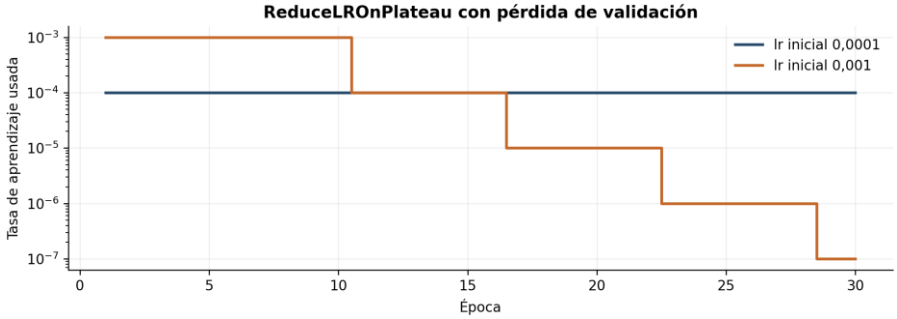


Figura 10. Tasa usada en cada época, bajadas de ReduceLROnPlateau .

11.2. Resolución de entrada Faster R-CNN v1

Como el rendimiento en objetos pequeños seguía siendo limitado (AP small = 0,1182), se aumentó la resolución de entrada de 800/1333 a 1024/1820 píxeles, manteniendo los pesos, tasa de aprendizaje de 0,001 y la aumentación. Así, un frame pasa de 1333 × 750 a 1820 × 1024 píxeles, lo que supone un 86 % más de píxeles. Debido al consumo de memoria, se utilizaron microlotes de 4 imágenes con acumulación de gradientes en dos pasos.

Mejor por AP50 en validación	lr001 800	lr001 1024	Cambio
AP50 todos	0,9433	0,9465	+0,32 pp
AP 0,50:0,95 todos	0,6302	0,6047	−2,55 pp
AP75 todos	0,7460	0,6990	−4,70 pp
AP50 difficult	0,6462	0,6282	−1,80 pp
AP small	0,1182	0,1886	+7,04 pp
FP / FN (conf. 0,5)	190 / 83	324 / 68	+134 / −15

Mejor por AP50 en validación	lr001 800	lr001 1024	Cambio
F1 (conf. 0,5)	0,9155	0,8840	−3,15 pp
Latencia por imagen	75,1 ms	121,6 ms	+61,9 %

Tabla 13. Resolución 800 frente a 1024 con el mismo procedimiento.

La Tabla 13 compara ambas resoluciones manteniendo el mismo procedimiento. Con 1024 píxeles mejora la detección de objetos pequeños (AP small pasa de 0,1182 a 0,1886) y se detectan 15 robots más, nueve de ellos difficult. Sin embargo, aumentan los falsos positivos en 134, empeoran el ajuste de las cajas y el F1 baja de 0,9155 a 0,8840. Además, la latencia aumenta un 61,9 %, pasando de 75,1 a 121,6 ms por imagen.

Aunque res1024 alcanza su mejor AP50 en la época 2 y su máximo AP 0,50:0,95 (0,6322) en la época 5, se mantiene el criterio de selección fijado previamente, basado en AP50. Por tanto, lr001 800px sigue siendo la referencia, ya que el aumento de resolución no mejora el rendimiento de forma uniforme. Por secuencias, lr001 mejora el AP50 en 7 de las 11 secuencias de validación, frente a 4 con res1024.

11.3. Comparación de arquitecturas

Se entrenaron los siete modelos del apartado 8 con pesos preentrenados de COCO y la cabeza de predicción adaptada a la detección de robots. Para cada modelo se seleccionó el checkpoint con mayor AP@0,5 en validación. La Tabla 14 recoge los resultados y la Figura 11 muestra la evolución del AP por época. Las gráficas interactivas, que permiten seleccionar la métrica, el modelo y el conjunto de datos, están disponibles en https://javiersaguar.github.io/PIDS-Proyecto2/bloque2/.

Modelo	Época	AP50 todos	AP50 normales	AP50 difficult	AP 0,5:0,95	F1 máx. (umbral)	ms/imagen	Min
Faster R-CNN v2	3	0,9513	0,9759	0,6422	0,6192	0,9361 (0,95)	99,5	151
RetinaNet v2	6	0,9447	0,9748	0,6213	0,6114	0,9264 (0,65)	67,3	102
Faster R-CNN v1	5	0,9426	0,9733	0,6471	0,6273	0,9305 (0,90)	75,1	106
RetinaNet v1	7	0,9396	0,9687	0,5994	0,6196	0,9229 (0,65)	72,2	98
Faster R-CNN MobileNet	3	0,9281	0,9597	0,6027	0,5789	0,9167 (0,55)	16,0	19
SSD300 VGG16	12	0,9008	0,9404	0,5199	0,5144	0,8926 (0,25)	12,5	19
SSDlite320	7	0,6823	0,7272	0,3233	0,3943	0,6839 (0,20)	4,2	6

Tabla 14. Siete arquitecturas con lr 0,001 en validación (672 imágenes, 1561 robots, 255 difficult). Época del mejor checkpoint, F1 máximo, ms por imagen de una en una y minutos de entrenamiento.

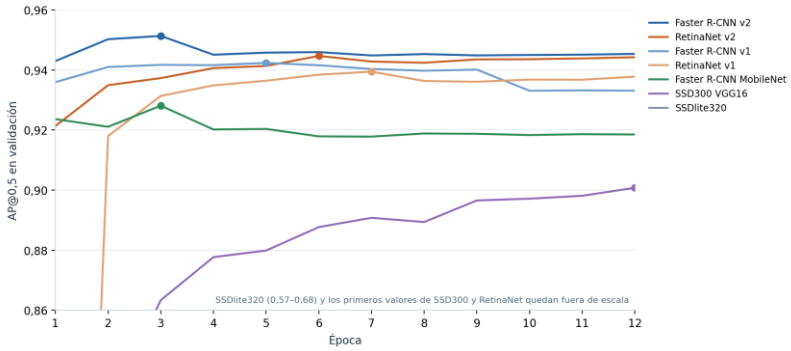


Figura 11. AP@0,5 en validación por época. El punto marca el checkpoint elegido.

Faster R-CNN v2 obtiene el mejor AP50 (0,9513), seguido de RetinaNet v2 (0,9447), Faster R-CNN v1 (0,9426) y RetinaNet v1 (0,9396). Los cuatro modelos con ResNet-50 presentan diferencias pequeñas, de apenas 1,2 puntos, por lo que no son concluyentes al haberse utilizado una sola semilla. Faster R-CNN alcanza su máximo entre las épocas 3 y 5 y después pierde entre 0,6 y 1 punto de AP mientras la pérdida de entrenamiento sigue bajando, lo que apunta a un sobreajuste temprano. RetinaNet comienza más despacio, pero mantiene un comportamiento más estable. SSD300 todavía mejora en la época 12, por lo que podría necesitar más entrenamiento, aunque no se probó con más épocas por mantener todos los entrenamientos unificando hiperparámetros.

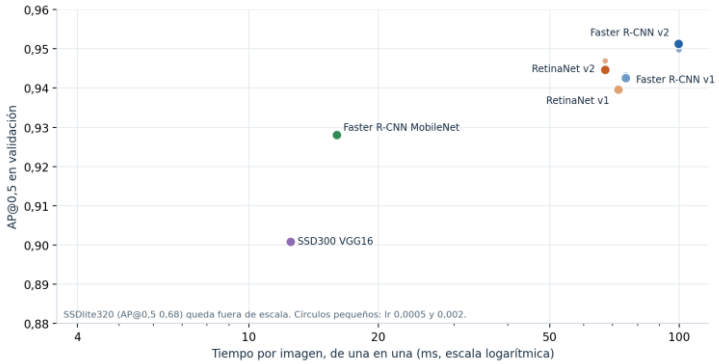


Figura 12. Precisión frente a tiempo por imagen.

Los modelos ligeros sacrifican precisión a cambio de velocidad. Faster R-CNN MobileNet pierde 2,3 puntos de AP50 respecto a la v2, pero es seis veces más rápido. SSD300 ofrece un AP50 de 0,9008, mientras que SSDlite320 baja a 0,6823, ya que su resolución dificulta detectar robots lejanos.

Los 13 entrenamientos suman 21,8 horas de GPU. Además, los objetos difficult siguen siendo el principal problema, con un AP@0,5 de 0,60–0,65, muy inferior al de los objetos normales. Cambiar de arquitectura no resuelve esta diferencia.

11.4. Tasa de aprendizaje y perfil de aumento

Modelo	lr	Mejor época	AP50 todos	AP50 difficult	AP 0,50:0,95	Min
Faster R-CNN v2	0,0005	5	0,9507	0,6361	0,6126	189
Faster R-CNN v2	0,0010	3	0,9513	0,6422	0,6192	151
Faster R-CNN v2	0,0020	1	0,9497	0,6223	0,6403	151
RetinaNet v2	0,0005	11	0,9451	0,6133	0,6170	126

Modelo	lr	Mejor época	AP50 todos	AP50 difficult	AP 0,50:0,95	Min
RetinaNet v2	0,0010	6	0,9447	0,6213	0,6114	102
RetinaNet v2	0,0020	6	0,9469	0,6246	0,6177	101
Faster R-CNN v1	0,0005	14	0,9420	0,6323	0,6104	132
Faster R-CNN v1	0,0010	5	0,9426	0,6471	0,6273	106
Faster R-CNN v1	0,0020	2	0,9434	0,6465	0,6024	105

Tabla 15. Variación de la tasa de aprendizaje en los tres modelos con mejor AP@0,5.

La tasa de aprendizaje apenas cambia el AP@0,5 (Tabla 15), pues las diferencias son inferiores a 0,22 puntos, así que pueden deberse a la semilla. Se mantiene 0,001 porque logra resultados similares a 0,0005 en menos épocas. Con 0,002, Faster R-CNN alcanza su máximo en una o dos épocas y después pierde entre 1,1 y 1,4 puntos, señal de un entrenamiento insuficiente.

También se comparó con los cambios de aumentación de datos para la robustez, con sombras, llovizna y más imágenes difficult. Los resultados son prácticamente iguales: AP@0,5 de 0,9426 frente a 0,9433. No se puede confirmar una mejora y se mantiene el aumento base.

11.5. Evaluación final en test

Se elige el modelo con mayor AP@0,5 en validación y el umbral con mejor F1.

Modelo	AP50 val.	AP50 test	Diff. test	Umbral	Precis.	Exhaust.	F1	FP / FN
Faster R-CNN v2	0,9513	0,9523	0,6921	0,95	0,9447	0,9332	0,9389	94 / 115
RetinaNet v2	0,9447	0,9457	0,6313	0,65	0,9409	0,9064	0,9233	98 / 161
Faster R-CNN v1	0,9426	0,9355	0,6057	0,90	0,9224	0,9250	0,9237	134 / 129
RetinaNet v1	0,9396	0,9362	0,6273	0,65	0,9226	0,9210	0,9218	133 / 136
Faster R-CNN MobileNet	0,9281	0,9240	0,5744	0,55	0,9145	0,9140	0,9142	147 / 148
SSD300 VGG16	0,9008	0,8797	0,4316	0,25	0,9128	0,8216	0,8648	135 / 307
SSDlite320	0,6823	0,6412	0,2742	0,20	0,7284	0,5985	0,6571	384 / 691

Tabla 16. Evaluación final en test, una vez por modelo y con el umbral fijado en validación.

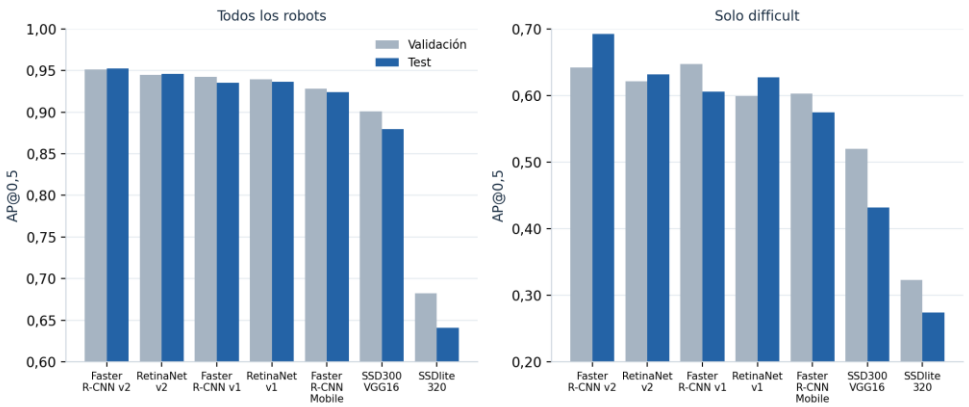


Figura 13. AP@0,5 en validación y en test.

Faster R-CNN v2 con lr 0,001 es el modelo final y alcanza un AP@0,5 de 0,9523 en test (0,9513 en validación). El AP es de 0,9658 en robots normales y 0,6921 en difficult. Con umbral 0,95 detecta 1606 de los 1721 robots, con una exhaustividad de 0,9332, precisión de 0,9447 y F1 de 0,9389. El mejor F1 en test sería 0,9413 con umbral 0,90, una diferencia mínima de 0,24 puntos. En general, los resultados de test coinciden con los de validación: las diferencias son pequeñas en los modelos ResNet-50, mientras que las SSD pierden más precisión (2,1 y 4,1 puntos).

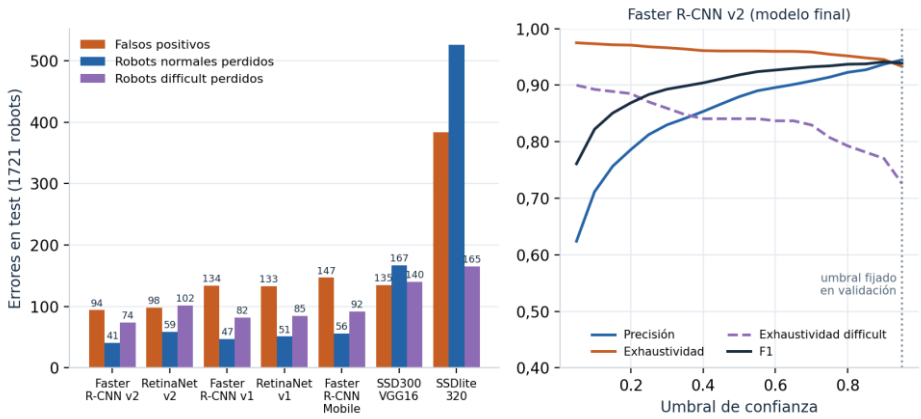


Figura 14. Errores en test con el umbral fijado en validación (izquierda) y curvas de umbral del modelo final (derecha)

Como se ve en la Figura 14, 74 de los 115 robots que no detecta el modelo final (64 %) son difficult, aunque representan solo el 16 % del total. Subir el umbral de 0,5 a 0,95 reduce los falsos positivos de 226 a 94, pero también baja la exhaustividad en difficult de 0,84 a 0,73.

Para la demo, Faster R-CNN MobileNet es una alternativa más rápida que obtiene un AP@0,5 de 0,9240 y un F1 de 0,9143, con 16 ms por imagen frente a 96,3 ms del modelo final.

12. Análisis cualitativo

Se puede apreciar en la Figura 15 dos predicciones de lr001 sobre validación, en las que se ve un acierto completo de tres detecciones correctas, incluida una con confianza 0,51 sobre un robot difficult, ósea que es una buena predicción. Las cajas verdes son aciertos y las rojas, detecciones sin pareja, por tanto, en el segundo caso pertenece a GR02_0001 de 2024, se confirmó que faltaban etiquetas (Figura 16), por lo que algunos falsos positivos no son errores reales del detector, sino que el conjunto de validación no estaba bien etqiuetado.



Figura 15. Predicciones del modelo lr001 sobre validación, acierto en PIDS2025_GR01_0010/frame_1566 (TP 3, FP 0, FN 0) y el caso con más errores, PIDS2024_GR02_0001/frame_0492 (TP 2, FP 4, FN 0).

Al revisar las predicciones, se distinguen errores del detector y etiquetas incompletas. En la Figura 16 aparecen cuatro robots, pero el XML solo incluye una caja, por lo que detectar los otros tres cuenta

como FP. Las etiquetas se mantienen para conservar la comparación. Esto limita la interpretación de los 190 FP de lr001 y los 324 de res1024, aunque no se sabe cuántos se deben a anotaciones omitidas.



Figura 16. Caso confirmado de anotación incompleta. La caja azul es la única del XML.

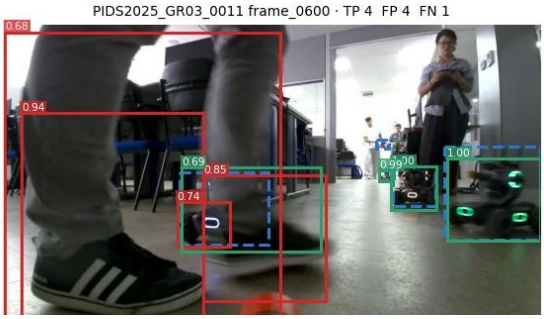


Figura 17. Res1024 en PIDS2025_GR03_0011/frame_0600: TP 4, FP 4 y FN 1 frente a las etiquetas..

Los fallos reales se concentran en tres casos. Primero, los objetos difficult, que acumulan 65 de los 83 FN de lr001 y 74 de los 115 del modelo final en test, aunque representan solo el 26,4 % de las cajas anotadas (375 de 1420). Segundo, los robots lejanos, con un AP small de 0,12–0,19, que son difíciles de detectar y están poco representados: no hay cajas originales de menos de 30 píxeles de ancho. Tercero, algunas secuencias especialmente difíciles: en test, PIDS2025_GR02_0009 pierde 40 de 204 robots y PIDS2025_GR04_0000 pierde 10 de 21; en validación, GR04_0003 de 2024 pierde sus seis objetos difficult. También hay falsos positivos sobre una pierna y cajas solapadas alrededor de un robot (Figura 17), que no se explican por etiquetas omitidas.

En test, subir el umbral de 0,5 a 0,95 reduce los falsos positivos de 226 a 94, pero baja la exhaustividad en difficult de 0,84 a 0,73. Se elige 0,95 porque obtuvo el mejor F1 en validación y queda a solo 0,24 puntos del óptimo en test (0,90).

12.1. Conclusiones y trabajo pendiente

El modelo final elegido es Faster R-CNN v2 con lr 0,001, cuyo checkpoint de la época 3 alcanza un AP@0,5 de 0,9523 en test (0,9658 en robots normales y 0,6921 en difficult) y un F1 de 0,9389 con el umbral 0,95 fijado en validación. Como alternativa rápida, podría estar Faster R-CNN MobileNet obtiene 0,9240 y tarda 16 ms por imagen, frente a los 96,3 ms del modelo final. Se elige este último por su mayor precisión, aunque habrá que comprobar si la latencia es adecuada al probarlo en el robot.

La arquitectura influye más que la tasa de aprendizaje o el aumento de datos y los difficult siguen siendo el principal problema, las SSD tienen dificultades con robots lejanos y Faster R-CNN sobreajusta pronto, por lo que el checkpoint se selecciona mediante validación.

Referencias

[1] labelImg. Herramienta de anotación PascalVOC. https://github.com/HumanSignal/labelImg

[2] Albumentations 1.4.18. Código de transformaciones. https://github.com/albumentations-team/albumentations/tree/1.4.18

[3] S. Ren et al. Faster R-CNN Towards Real-Time Object Detection with Region Proposal Networks. 2015. https://arxiv.org/abs/1506.01497

[4] torchvision 0.19. Faster R-CNN ResNet-50 FPN. https://docs.pytorch.org/vision/0.19/models/generated/torchvision.models.detection.fasterrcnn_resnet50_fpn.html

[5] Repositorio del grupo. Datos, criterios, notebooks y resultados. https://github.com/javiersaguar/PIDS-Proyecto2

[6] Web del proyecto interactiva. https://javiersaguar.github.io/PIDS-Proyecto2/

[7] Notebook del EDA ejecutado (eda.ipynb). https://htmlpreview.github.io/?https://github.com/javiersaguar/PIDS-Proyecto2/blob/main/ENTREGA%201/WEEK1/eda.html

[8] Notebook del aumento de datos ejecutado (data_aug.ipynb). https://htmlpreview.github.io/?https://github.com/javiersaguar/PIDS-Proyecto2/blob/main/ENTREGA%201/WEEK1/data_aug.html

[9] T.-Y. Lin et al. Focal Loss for Dense Object Detection. 2017. https://arxiv.org/abs/1708.02002

[10] W. Liu et al. SSD: Single Shot MultiBox Detector. 2016. https://arxiv.org/abs/1512.02325

[11] K. He et al. Deep Residual Learning for Image Recognition. 2016. https://arxiv.org/abs/1512.03385

[12] T.-Y. Lin et al. Feature Pyramid Networks for Object Detection. 2017. https://arxiv.org/abs/1612.03144

[13] A. Howard et al. Searching for MobileNetV3. 2019. https://arxiv.org/abs/1905.02244

[14] K. Simonyan y A. Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. 2015. https://arxiv.org/abs/1409.1556

[15] T.-Y. Lin et al. Microsoft COCO: Common Objects in Context. 2014. https://arxiv.org/abs/1405.0312

[16] Y. Li et al. Benchmarking Detection Transfer Learning with Vision Transformers. 2021. https://arxiv.org/abs/2111.11429

[17] M. Sandler et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks. 2018. https://arxiv.org/abs/1801.04381

[18] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. 2019. https://arxiv.org/abs/1912.01703

Los originales, XML, exclusiones y manifiestos están en el repositorio. Los HTML del EDA son material complementario; las figuras del cuerpo son exportaciones estáticas. Los experimentos y tablas de esta

versión se han medido en validación, salvo las comprobaciones de integridad del aumento y la evaluación final, que se hizo en test una sola vez por modelo.